

UniApp 中 navigator 组件与路由跳转全解析

UniApp 中 navigator 组件与路由跳转全解析

在 uni-app 中，页面跳转主要有两种核心方式：`<navigator>` 组件（声明式跳转）和 路由 API（编程式跳转），二者均基于 pages.json 中注册的路由规则，以下是详细用法、场景及注意事项：

一、前置准备：路由配置（pages.json）

所有需要跳转的页面必须在 `pages.json` 的 `pages` 数组中注册，否则跳转失效。

Code block

```
1  {
2    "pages": [
3      // 首页（默认启动页，数组第一个元素）
4      {
5        "path": "pages/index/index",
6        "style": { "navigationBarTitleText": "首页" }
7      },
8      // 目标页面
9      {
10       "path": "pages/detail/detail",
11       "style": { "navigationBarTitleText": "详情页" }
12     },
13     {
14       "path": "pages/login/login",
15       "style": { "navigationBarTitleText": "登录页" }
16     }
17   ],
18   "globalStyle": {
19     "navigationBarBackgroundColor": "#fff",
20     "navigationBarTextStyle": "black"
21   }
22 }
```

二、声明式跳转：<navigator> 组件

通过标签直接实现跳转，类似 HTML 的 `<a>` 标签，适合简单的点击跳转场景。

1. 核心属性

--	--	--

属性名	类型	说明
url	String	跳转目标页面路径（必填），支持相对路径/绝对路径，可带参数（如 <code>pages/detail/detail?id=10</code> ）
open-type	String	跳转方式（核心），默认 <code>navigate</code>
delta	Number	仅 <code>open-type="navigateBack"</code> 时有效，返回的页面数，默认 1
hover-class	String	点击时的样式类，默认 <code>navigator-hover</code>
redirect	Boolean	已废弃，改用 <code>open-type="redirect"</code>
navigate-back	Boolean	已废弃，改用 <code>open-type="navigateBack"</code>

2. 常用 open-type 取值及场景

open-type 值	功能描述	对应 API	页面栈行为
navigate	保留当前页，跳转到新页（可返回）	uni.navigateTo	页面栈 +1
redirect	关闭当前页，跳转到新页（不可返回当前页）	uni.redirectTo	页面栈 替换当前页
switchTab	跳转到 tabBar 页面，关闭其他所有非 tabBar 页	uni.switchTab	清空非 tabBar 页，tabBar 页
reLaunch	关闭所有页面，跳转到新页	uni.reLaunch	清空页面栈，新页
navigateBack	返回上一页/多级页面	uni.navigateBack	页面栈 -delta（默

3. 实战示例

Code block

```
1 <template>
2   <view class="nav-demo">
```

```
3      <!-- 1. 普通跳转（保留当前页，可返回） -->
4      <navigator url="/pages/detail/detail?id=10&name=uniapp" open-
type="navigate">
5          跳转到详情页（带参数）
6      </navigator>
7
8      <!-- 2. 重定向（关闭当前页，不可返回） -->
9      <navigator url="/pages/login/login" open-type="redirect">
10         跳转到登录页（关闭当前页）
11     </navigator>
12
13     <!-- 3. 跳转到 tabBar 页面（需在 pages.json 配置 tabBar） -->
14     <navigator url="/pages/index/index" open-type="switchTab">
15         返回首页（tabBar）
16     </navigator>
17
18     <!-- 4. 关闭所有页面，跳转到新页 -->
19     <navigator url="/pages/index/index" open-type="reLaunch">
20         重启到首页（清空所有页）
21     </navigator>
22
23     <!-- 5. 返回上一页（delta=2 表示返回2级） -->
24     <navigator open-type="navigateBack" delta="1">
25         返回上一页
26     </navigator>
27 </view>
28 </template>
```

三、程式跳转：路由 API

通过调用 uni 提供的路由方法实现跳转，适合需要先执行逻辑（如表单验证、数据请求）再跳转的场景。

1. 核心 API 详解

(1) uni.navigateTo({})

保留当前页面，跳转到应用内的某个页面（**不允许跳转到 tabBar 页面**），可通过 `uni.navigateBack` 返回。

Code block

```
1  // 触发跳转（如点击按钮）
2  handleToDetail() {
3      // 携带参数跳转
4      uni.navigateTo({
5          url: '/pages/detail/detail?id=10&name=uniapp', // 参数拼接在url后
```

```

6      success: () => {
7          console.log('跳转成功');
8      },
9      fail: (err) => {
10         console.log('跳转失败', err);
11     }
12 });
13 }
14
15 // 目标页面 (detail.vue) 接收参数
16 onLoad(options) {
17     // options 是跳转时携带的参数对象
18     console.log(options.id); // 10
19     console.log(options.name); // uniapp
20 }

```

(2) uni.redirectTo({})

关闭当前页面，跳转到应用内的某个页面（**不允许跳转到 tabBar 页面**），无法返回当前页。

Code block

```

1  handleRedirect() {
2      uni.redirectTo({
3          url: '/pages/login/login',
4          success: () => {
5              console.log('重定向成功, 当前页已关闭');
6          }
7      });
8  }

```

(3) uni.switchTab({})

跳转到 tabBar 页面，并关闭其他所有非 tabBar 页面（**只能跳转到配置过 tabBar 的页面**）。

Code block

```

1  // pages.json 需先配置 tabBar
2  // "tabBar": { "list": [{ "pagePath": "pages/index/index", "text": "首页" }] }
3  handleToTabBar() {
4      uni.switchTab({
5          url: '/pages/index/index',
6          success: () => {
7              console.log('跳转到tabBar首页');
8          }
9      });
10 }

```

(4) uni.reLaunch({})

关闭所有页面，跳转到应用内的某个页面（可跳转到任意页面，包括 tabBar），常用于退出登录后跳转到首页。

Code block

```
1  handleReLaunch() {
2    uni.reLaunch({
3      url: '/pages/index/index',
4      success: () => {
5        console.log('所有页面已关闭，跳转到首页');
6      }
7    });
8  }
```

(5) uni.navigateBack({})

关闭当前页面，返回上一页面或多级页面，通过 `delta` 指定返回层级。

Code block

```
1  // 返回上一页（默认 delta=1）
2  handleBack() {
3    uni.navigateBack({
4      delta: 1,
5      success: () => {
6        console.log('返回上一页');
7      }
8    });
9  }
10
11 // 返回2级页面
12 handleBackTwo() {
13   uni.navigateBack({
14     delta: 2
15   });
16 }
```

2. 复杂参数传递（对象/数组）

若需传递复杂数据（如对象、数组），直接拼接 URL 会报错，需先转 JSON 字符串，目标页再解析：

Code block

```
1  // 跳转页
```

```
2  handleToDetail() {
3    const complexData = {
4      id: 10,
5      info: { title: 'uniapp教程', content: '路由跳转' },
6      tags: ['vue', 'uni-app']
7    };
8    // 转JSON字符串并编码（避免特殊字符问题）
9    const dataStr = encodeURIComponent(JSON.stringify(complexData));
10   uni.navigateTo({
11     url: `/pages/detail/detail?data=${dataStr}`
12   });
13 }
14
15 // 目标页 (detail.vue)
16 onLoad(options) {
17   // 解码并解析JSON
18   const complexData = JSON.parse(decodeURIComponent(options.data));
19   console.log(complexData.info.title); // uniapp教程
20 }
```

四、页面栈与生命周期

uni-app 的页面跳转基于**页面栈**管理，不同跳转方式会影响页面生命周期：

跳转方式	原页面生命周期	新页面生命周期	页面栈变化
navigateTo	onHide	onLoad → onShow	入栈
redirectTo	onUnload	onLoad → onShow	替换
switchTab	onHide	onShow（仅触发，无onLoad）	清空非tabBar页
reLaunch	onUnload	onLoad → onShow	清空重入
navigateBack	onUnload	onShow（返回的页面）	出栈

注意： `switchTab` 跳转到已打开的 tabBar 页面时，目标页只会触发 `onShow`，不会触发 `onLoad`，需通过 `uni.$on/uni.$emit` 传递实时数据。

五、常见问题与注意事项

1. 路径格式：

- 绝对路径：以 `/` 开头（推荐），如 `/pages/detail/detail`；

- 相对路径：如 `../detail/detail`，需注意当前页面层级，易出错。

2. tabBar 页面跳转限制：

- `navigateTo` / `redirectTo` 无法跳转到 tabBar 页面，必须用 `switchTab` / `reLaunch`；
- `switchTab` 的 url 不能带参数，需通过全局变量（如 `getApp().globalData`）或缓存传递。

3. 页面返回传值：

若需从详情页返回首页并传值，可通过 `uni.$emit` / `uni.$on` 或缓存：

Code block

```
1  // 详情页（返回时传值）
2  handleBackWithData() {
3    // 触发全局事件
4    uni.$emit('detailBack', { id: 10, status: 'success' });
5    uni.navigateBack();
6  }
7
8  // 首页（监听事件）
9  onShow() {
10   uni.$on('detailBack', (data) => {
11     console.log('详情页返回数据', data);
12   });
13 }
14 // 注意：页面卸载时需移除监听，避免内存泄漏
15 onUnload() {
16   uni.$off('detailBack');
17 }
```

4. 跳转失败排查：

- 检查 `pages.json` 是否注册目标页面；
- 检查 url 路径是否拼写错误（区分大小写）；
- 避免循环跳转（如 A 跳 B，B 又跳 A）；
- 小程序端需注意路径层级不超过 10 层。

5. 小程序分包跳转：

若目标页面在分包中，url 需包含分包路径，如 `/packageA/pages/subDetail/subDetail`。

六、总结

跳转场景	推荐方式	核心 API/组件属性
普通跳转（可返回）	声明式/编程式	navigator(open-type=navigate) / uni.navigateTo
跳转到 tabBar 页面	声明式/编程式	navigator(open-type=switchTab) / uni.switchTab
关闭当前页跳转	声明式/编程式	navigator(open-type=redirect) / uni.redirectTo
重启应用（清空所有页）	编程式	uni.reLaunch
返回上一页/多级页	声明式/编程式	navigator(open-type=navigateBack) / uni.navigateBack
先执行逻辑再跳转	编程式	所有 API

通过合理选择跳转方式，结合页面栈和生命周期特性，可高效实现 uni-app 跨端页面跳转，同时避免路径错误、参数传递失败等常见问题。

（注：文档部分内容可能由 AI 生成）